

Computer Science Notes

Revision Guide

Jintian-JTST

A note presented for Zoe's
computer science classes

Acknowledgements

Special thanks to $\LaTeX$, www.overleaf.com, and **Zoe's support** of writing this document on her class. Without this, I would not be able to finish this revision without my after-school time.

$\LaTeX$ is a kind of programming language, which provides a wide range of codes for academic students to input equations, lemmas, theorems, definitions easier while comparing to WPS or Office and Markdown.

Some people may have seen me editing this notes before, but this is a continuously updating program, which my goal is to collect all the points on the coursebook and what my teacher has taught me, and serve them as dishes to my guests, yeah, the readers, you guys in front of this screen or paper. If you do have any suggestions, please contact me through WeChat or e-mail, also if you do have any criticisms, remind me in either way.

Some may be confused about *what are the differences between different colors, fonts, and bold ones?* To answer this, I just want to attract your attention to some places, which some of them are important, some are difficult, some are for division between successive parts, some are just for fun.

Lastly, I **did add** some of my own views and opinions into this note, and I want to claim that if you have any uncertainties between the coursebook and this note, please regard this one as trash! **Always make sure that your answer follows the syllabus!**

Enjoy your revision to any examinations and wish you a nice score in the exam!

E-mail contact: wangjintianlovemc@163.com

@Jintian-JTST

$\LaTeX$

Contents

1	Number Presentation	2
1.1	Binary System	2
1.2	Multimedia	4
1.3	File Compression	5
2	Hardware	7
2.1	Input and Output Devices	7
2.2	Memory	9
2.3	Logic Gate	10
3	Processor fundamentals	11
3.1	Components in and out of CPU	13
3.2	Fetch, Decode, Execute Cyle	15
3.3	Interrupts	16
3.4	Assembly language	17
4	System Software	19
4.1	Operating System	19
4.2	Utility Software, Libraries, and Translators	20

Chapter 1

Number Presentation

1.1 Binary System

Definition 1.1.1 (Denary System). Also called Decimal System. Denary is the number system we use, **base 10**.

Each digit in a number has a different place value, working from the RHS value increases in the powers of 10.

Definition 1.1.2 (Binary). Base two number system based on the values 0 and 1 only.

128	64	32	16	8	4	2	1
0	1	1	1	0	1	0	1

The binary value above is equal to:

$$64 + 32 + 16 + 4 + 1 = 117$$

So

$$117_{10} = 01110101_2$$

Definition 1.1.3 (Bit, Byte, and Nibble). Abbreviation for binary digit. A binary digit (1 or 0) is known as a bit. Short for **B**inary digit. A group of 8 bits treated as a single unit is called a **byte**. A group of 4 bits is called a **nibble**.

Theorem 1.1.1 (Overflow). A condition when **the result of a calculation** is too large to **fit into** the number of bits **defined for storage**.

The applications of Hexadecimal Representation

- HTML
- Error code
- MAC address
- Web address

Definition 1.1.4 (Binary-coded decimal). BCD. Number system that uses 4bits to represent each denary digit. For example, the BCD code of 12345 is:

$$00010010001101000101_{BCD}$$

Several applications of BCD

1. Barcode
2. Digital screen
3. Represent decimal fractions

Pros and Cons of BCD

Advantage Allows **easy conversion** between decimal and binary form.

Disadvantage Wasteful. States between 1010_2 and 1111_2 are not used.

Definition 1.1.5 (One's Complement). Each binary digit in a number is **reversed** to allow **both negative and positive numbers** to be represented. e.g. The one's complement form of 01101011_2 is:

$$10010100_2$$

Definition 1.1.6 (Two's Complement). Each binary digit is **reversed** and **1 is added in right-most position** to produce another method of **representing positive and negative numbers**. e.g. The two's complement form of 01101011_2 is:

$$10010100_2 + 1_2 = 10010101_2$$

Decimal Prefix:

Decimal prefix name	Symbol used	Factor
kilo	K	10^3
mega	M	10^6
giga	G	10^9
tera	T	10^{12}

Binary Prefix:

Binary prefix name	Symbol used	Factor
kibi	Ki	2^{10}
mebi	Mi	2^{20}
gibi	Gi	2^{30}
tebi	Ti	2^{40}

Definition 1.1.7 (ASCII code). Coding system for all the characters on a keyboard and control codes.

Notes ASCII is short for *America Standard Code for Information Interchange*. It is a 7-bit code, includes 128 characters.

Definition 1.1.8 (Unicode). Coding system which represents all the languages of the world (first 128 characters are the same as ASCII code).

1.2 Multimedia

Definition 1.2.1 (Bitmap). System that uses pixels to make up an image.

Definition 1.2.2 (Vector graphics). Images that use 2D points to describe lines and curves and their properties that are grouped to form geometric shapes.

Definition 1.2.3 (Pixel). The **smallest identifiable** component of a bitmap image. Defined by two properties: **position** and **color**.

Definition 1.2.4 (Color Depth). The number of bits used to represent one pixel. n-bit color depth image can show at most 2^n colors.

Image Resolution refers to the number of pixels that make up an image.

Screen Resolution refers to the number of horizontal pixels and the number of vertical pixels that make up a screen display.

Lemma 1.2.1 (Calculating resolutions). The size of an image refers to its height, width, and color depth. **Size=Number of Pixels $\times$ Color Depth = Image Size.** e.g. Calculate a 1080p 24 color-depth image size.

$$\frac{1080 \times 1920 \times 24}{8 \times 2^{20}} = 5.93MiB$$

Definition 1.2.5 (Sampling resolution). Number of bits used to represent sound amplitude (also known as bit depth).

Definition 1.2.6 (Sampling rate). Number of sound samples taken per second.

To decide how often you are going to sample the analogue signal is called the **sampling rate**.

- (1) too little samples stored on the computer will sound **very distant** from the one being recorded.
- (2) too many samples and sound stored will **resemble the original sound** but having store each of the samples means you'll get very **large file sizes**.

Vector graphic images	Bit-map images
made up of geometric shapes which require definition/attributes	made up of tiny pixels of different colours
to alter/edit the design, it is necessary to change each of the geometric shapes	possible to alter/edit each of the pixels to change the design of the image
they do not require large file size since it is made up of simple geometric shapes	because of the use of pixels (which give very accurate designs), the file size is very large
because the number of geometric shapes is limited, vector graphics are not usually realistic	since images are built up pixel by pixel, the final image is usually very realistic
file formats are usually .svg, .cgm, .odg	file formats are usually .jpeg, .bmp, .png

1.3 File Compression

Definition 1.3.1 (Lossless file compression). File compression method where the original file can be restored following decompression.

Definition 1.3.2 (Lossy file compression). File compression method where parts of the original file cannot be recovered during decompression, so some of the original detail is lost.

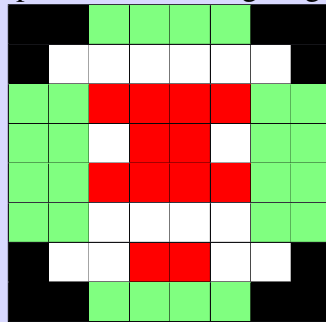
Definition 1.3.3 (Run length encoding (RLE)). A lossless file compression technique used to reduce text and photo files in particular

Example

1. Consider the text string *aaaaabbbbccddddd*. Assuming each character requires 1byte, then this string needs 16 bytes. Using ASCII code, then the string can be coded as follows:

05, 97, 04, 98, 02, 99, 05, 100

2. Use run-length encoding to compress the following diagram:



Given that for a single box:

Square color	Red	Green	Blue
black box	0	0	0
white box	255	255	255
green box	0	255	0
red box	255	0	0

This produce the following data:

2B4G3B6W1B2G4R4G1W2R1W4G4R4G4W2G1B2W2R2W3B4G2B

The original image needs 3 bytes to present one square. Therefore, it needs $8 \times 8 \times 3 = 192$ bytes. But after compression file only has 92 bytes, which the rate of compression is about 52%.

CHAPTER 1. NUMBER PRESENTATION

American Standard Code for Information Interchange form

DEC	HEX	Char	DEC	HEX	Char	DEC	HEX	Char
32	20	space	64	40	@	96	60	`
33	21	!	65	41	A	97	61	a
34	22	”	66	42	B	98	62	b
35	23	#	67	43	C	99	63	c
36	24	\$	68	44	D	100	64	d
37	25	%	69	45	E	101	65	e
38	26	&	70	46	F	102	66	f
39	27	,	71	47	G	103	67	g
40	28	(	72	48	H	104	68	h
41	29	)	73	49	I	105	69	i
42	2a	*	74	4a	J	106	6a	j
43	2b	+	75	4b	K	107	6b	k
44	2c	,	76	4c	L	108	6c	l
45	2d	-	77	4d	M	109	6d	m
46	2e	.	78	4e	N	110	6e	n
47	2f	/	79	4f	O	111	6f	o
48	30	0	80	50	P	112	70	p
49	31	1	81	51	Q	113	71	q
50	32	2	82	52	R	114	72	r
51	33	3	83	53	S	115	73	s
52	34	4	84	54	T	116	74	t
53	35	5	85	55	U	117	75	u
54	36	6	86	56	V	118	76	v
55	37	7	87	57	W	119	77	w
56	38	8	88	58	X	120	78	x
57	39	9	89	59	Y	121	79	y
58	3a	:	90	5a	Z	122	7a	z
59	3b	;	91	5b	[	123	7b	{
60	3c	i	92	5c	\$	124	7c	
61	3d	=	93	5d	]	125	7d	}
62	3e	¿	94	5e	^	126	7e	~
63	3f	?	95	5f	_	127	7f	delete

Extension Readings

ASCII is originally designed for communication between computers and peripherals, which represents a range of characters, including uppercase and lowercase letters, digits, punctuation marks, and control characters.

ASCII encoding assigns a unique numerical value to each character, making it possible to represent text in a machine-readable format. Nowadays, with the advent of Unicode, which supports a much broader range of characters, ASCII has been largely superseded for international text encoding.

Chapter 2

Hardware

2.1 Input and Output Devices

Capacitive Touch Screen

- Made up of *many layers of glass* that act like a capacitor creating electric field between the layers in layers.
- When top glass layer is touched, the *electric current* changes and the coordinates where the screen was touched are determined by an on board microprocessor.

Benefits

- Medium *cost* technology.
- Screen *visibility* is good even in strong sunlight.
- Permits *multi-touch* capability.
- Screen is very durable.

Drawbacks

- Only allows use of bare fingers as the form of input.

Resistive Touch Screen

This type of touchscreen is out of date.

- Made up of *an upper layer of polyester and a bottom layer* of glass.
- When top polyester - layer is touched, the top layer and bottom layer *complete a circuit*.
- Signals are then sent out, which are interpreted by a microprocessor and the calculations determine the coordinates of where the screen was touched.

Benefits

- Relatively inexpensive technology.
- Possible to use bare fingers, gloved fingers or stylus to carry out an input operation.

Drawbacks

- Screen *visibility* is poor in strong sunlight.
- Does not permit *multi-touch* capability.
- Screen durability is only fair.

CHAPTER 2. HARDWARE

Lemma 2.1.1 (Microphone). When a microphone picks up sounds, a **diaphragm** vibrates producing an analogue electric signal. The signal is converted into a digital signal which can be processed by computer through *Analogue to Digital Converter*.

Definition 2.1.1 (Laser Printer). A non-impact printer, which produce **high quality text and graphics**, using *toners*.

HOW IT WORKS

1. The drum is given an *electric charge*.
2. The drum starts to revolve step by step.
3. A laser beam is directed by the mirror and lens assembly to a sequence of positions across the width of the drum.
4. At each position the laser is either *switched off* to leave the charge on the drum or *switched on* to discharge the position.
5. These process repeats until a **full-page electrostatic image** has been created.
6. The drum is coated with a charged toner that only sticks to positions where the drum has been discharged.
7. The drum rolls over a sheet of paper which is initially given an electric charge.
8. The sheet of paper is discharged and then is passed through heated rollers to fuse the toner particles and seat the image on the paper surface.
9. The drum is **discharged** before the process starts again for the next page.

2.2 Memory

Definition 2.2.1 (Hard Disk). A **non-volatile storage** device that stores digitally data on rapidly *rotating platters* with magnetic surfaces.

Reading

- ≪≪≪ Platters spin around the spindle.
- ≪≪≪ Data is requested to be read from a specific area.
- ≪≪≪ Actuator arm moves the read head to that track.
- ≪≪≪ Once the data sector that is required has spun under the read head, data is read.
- ≪≪≪ Data read is sent from the IDE connector to main memory.

Writing

- ≫≫≫ Platters spin around the spindle.
- ≫≫≫ Data is sent from main memory to hard disk using IDE connector.
- ≫≫≫ Data is requested to be written into a specific area.
- ≫≫≫ Actuator arm moves the write head to that track.
- ≫≫≫ Once the data sector that is required has spun under the read head, data is written.

Definition 2.2.2 (Random Access Memory). Primary memory unit that can **be written to and read from**. Also called RAM.

There are two types of RAM, *Dynamic RAM* and *Statics RAM*.

DRAM	SRAM
consists of transistors and capacitors	uses flip-flops to hold each bit of memory
needs to be constantly refreshed	doesn't need to be constantly refreshed
cheap, high memory capacity	expensive, low memory capacity
main memory is constructed from DRAM	processor memory cache makes use SRAM
consumes more power under reasonable levels of access, as it needs to be refreshed	if accessed at a high frequency, power usage can exceed that of DRAM

Definition 2.2.3 (Read Only Memory). Primary memory unit that can only **be read from**. Also called ROM. The main difference between RAM and ROM is that data in ROM will not be removed when *switching off the computer* but RAM will.

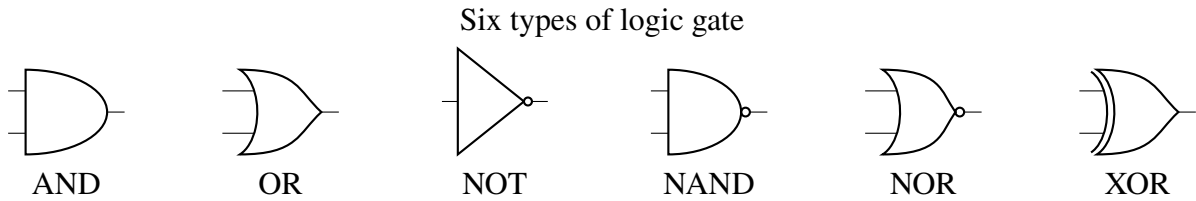
ROM -the simplest type. Programs are installed when *manufacturing* the ROM.

Programmable ROM -no data when manufacturing, but written in when the manufacturer supplied them to a system builder. Those ROM are mainly used for storing system programs that won't change when put into use. The program or data that once installed into it *cannot be changed*.

Erasable Programmable ROM -can erased by the ultra-violet light. So when erasing, ROM should be taken out from the circuit. Then new data can be written into the ROM.

Electric Erasable Programmable ROM -similar to EPROM, but can be erased by *electricity*. Much more convenient to other ROMs.

2.3 Logic Gate



Notice About the **truth table** of various gates, please check you memory by yourself and your coursebook. (I don't believe that somebody would forget this!)

Fundamental Identities

Lemma 2.3.1 (Commutative Law, Associative Law, Idempotent Law, and Annihilator Law).

Commutative Law	$x + y = y + x$	$x \cdot y = y \cdot x$
Associative Law	$(x + y) + z = x + (y + z)$	$x \cdot (y \cdot z) = (x \cdot y) \cdot z$
Idempotent Law	$x + x = x$	$x \cdot x = x$
Annihilator Law	$x + 1 = 1$	$x \cdot 1 = x$

Lemma 2.3.2 (Complement Law). A term *or*'ed with its complement equals 1 and a term *and*'ed with its complement equals 0.

$$x + \bar{x} = 1$$

$$x \cdot \bar{x} = 0$$

Theorem 2.3.3 (Absorptive Law, and Distributive Law). Absorptive Law: Complex expressions can be reduced to a simpler ones by absorbing like terms. Distributive Law: It's OK to multiply or factor-out an expression.

$x + x \cdot y = x$ $x + \bar{x}y = x + y$ $x(x + y) = x$	$x \cdot (y + z) = xy + xz$ $(x + y) \cdot (p + q) = xp + xq + yp + yq$ $(x + y) \cdot (x + z) = x + yz$
---	--

Theorem 2.3.4 (**DeMorgan's Law**). An *or/and* expression that is negated is equal to the *and/or* of the negation of each term.

$$\overline{x + y} = \bar{x} \cdot \bar{y}$$

$$\overline{x \cdot y} = \bar{x} + \bar{y}$$

Relationship between XOR and XNOR $x \odot y = \overline{x \oplus y} = x \oplus \bar{y} = \bar{x} \oplus y$

Chapter 3

Processor fundamentals

Some words I would like to say in front of stepping into the notes of this chapter

This chapter is mainly about the most basic hardware system in a CPU, which is pretty abstract and difficult for students to understand unless you've done an extreme research on it. I'm not here to show how high the level I have, but it is true that I would like to show you another aspect of understanding this part. Some of you may have learnt it in Pre level, but what I want to tell you is that those studied in Pre level was too easy to applied in AS level, and unnecessary to mention about the real world ones. So, try to understand my method, but be careful that my way is **NOT SERIOUS**, which only provides you a way to understand it, instead of using them as answers to the exams. If you want to, then **remember the syllabus and coursebooks as your best teacher**.

By the way, if you see any black boxes like the one used here currently, and provided you just want to absorb the original knowledge from the coursebook, please ignore them and jump to the end of those boxes to see the real key points. Those black boxes are written by author myself, which have no use to your exams, but I'm very sure that you will get a different understanding of the concepts if you read those additional texts.

Nowadays there are two kinds of computer structures, *Von Neumann* and *Harvard*. More than 80% of the CPUs are made in **Von Neumann structure**, which was developed in mid-1940s.

Differences in Von Neumann and Harvard

The Von Neumann architecture has a single memory that stores both instructions and data. This means that the memory is shared between the program instructions and the data being processed. The Harvard architecture has separate memory spaces for instructions and data. This means that the program instructions and the data are stored in distinct memory units.

Features of Von Neumann architecture

- A central processing unit, you may call it **CPU or processor**.
- A processor able to **access memory directly**.
- Computer memories that could store **programs** as well as **data**.
- Programs made up of a **number of instructions** can be executed in sequences.

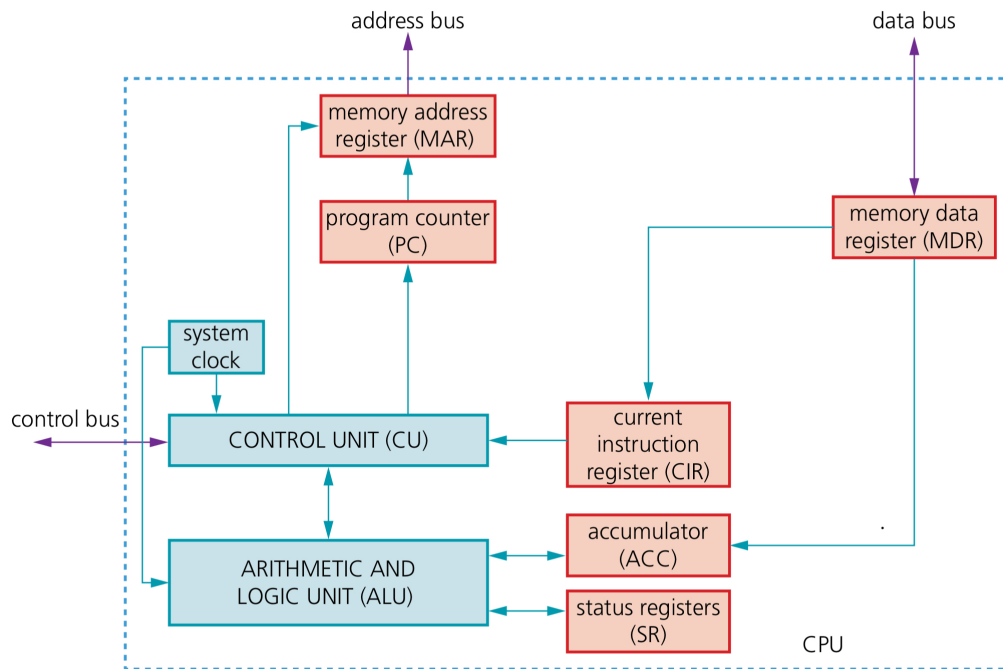


Figure 3.1: Representation of Von Neumann architecture.

Change in Von Neumann Architecture

To be honest, the earliest version of Von Neumann structure is not actual the same that are using today. If your teacher had told you, that junior architecture always uses CPU, no matter you write something to your memory or multiply two numbers together. This may cause great burden on the processor, which everything, even the most easiest task that can be done by a lower level chip, should be done by the central one, which drag the period down. Later the researcher found this problem, and add a little bit change to it, that everything should go across the memory, and some basic tasks can be processed by memory itself, as a high proportional of easy instructions are about memories.

Figure 3.1 shows a simple representation of Von Neumann architecture. From the diagram we can clearly see the connection between each units and buses.

3.1 Components in and out of CPU

Main Components inside

Definition 3.1.1 (Arithmetic Logic Unit). Component in the processor which carries out all arithmetic and logical operations.

Definition 3.1.2 (ACCumulator). A temporary register used when carrying out ALU calculations.

Definition 3.1.3 (Control Unit). It ensures synchronisation of data flow and programs throughout the computer by sending out control signals along the control bus. In simple words, it reads instructions and interprets them, then control other units through control bus.

Definition 3.1.4 (System Clock). It produces timing signals on the control bus to ensure synchronisation takes place.

Definition 3.1.5 (Immediate Access Store). It holds all data and programs needed to be accessed by the control unit.

Register	Abbreviation	Function/purpose of register
current instruction register	CIR	stores the current instruction being decoded and executed
index register	IX	used when carrying out index addressing operations (assembly code)
memory address register	MAR	stores the address of the memory location currently being read from or written to
memory data/ buffer register	MDR/MBR	stores data which has just been read from memory or data which is about to be written to memory (sometimes referred to as MBR)
program counter	PC	stores the address where the next instruction to be read can be found
status register	SR	contain bits which can be set or cleared depending on the operation (for example, to indicate overflow in a calculation)

Figure 3.2: Common Registers

IX and other registers will be introduced later, we now here define SR.

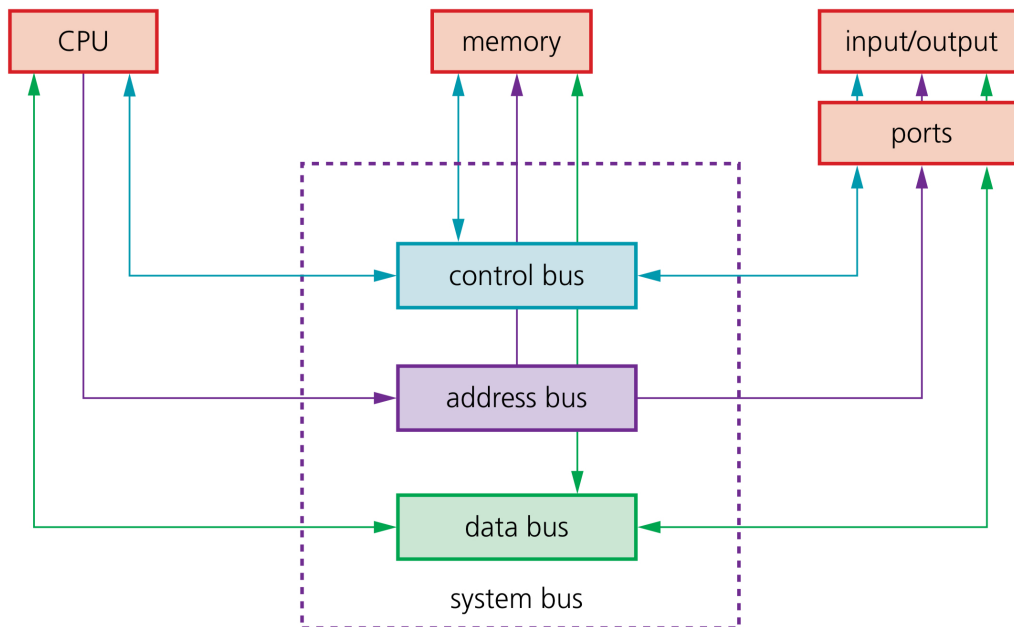
Status Register

Status Register is used when an instructions requires the status of cpu while processing.

- Carry flag is true while we needs to CARRY the result of the addition instruction.
- Negative flag is true if the result is NEGATIVE- in two's complement.
- oVerflow flag is true if there is an overflow.
- Zero flag is true if the result is zero.

ACC

Here are some ways for you to feel it: It is a register. All the data that is going to enter ALU will be stored in it temporarily. After each calculation it stores the output of ALU.



Attention: Only address bus is **unidirectional** bus. The other two buses are **bidirectional**. If you don't know those words, check them by yourself to leave deeper impression.

Clock

When I was a child and was going to study CPU structure, I cannot understand clock. But now I think I can hand out something interesting. As for users, like watching a 1080p 3color 8depth 60Hz screen, you can see how much data will be transferred each second, $1080 \times 3 \times 8 \times 60 = 1.55 \times 10^6$ bits, but in a small processor there will only be 16 to 32 bits on the bus or even less, so it is impossible to transfer all the data at the same time. In this way, we need to divide data so that is enough in order not to stuck on a single frame. 1.55 million bits are transferred per second, but 16 bits per time, so at least we need to transfer $1.55 \times 10^6 / 16 = 96875$ times per second, and that is the frequency of the clock, 96kHz of data bus. Remember, not only the diagrams of the screen are waited for transfer, but also other stuffs such as sound, cameras, printers. So nowadays that clock frequency is set to more than 3GHz, but cannot be quicker because of physics. That light travels 3×10^8 meters per second, which in a single period $13 \times 10^8 = 3.33 \times 10^{-10}$ seconds, which only travels $3.33 \times 10^{-10} \times 3 \times 10^8 = 10$ cm, an about distance in motherboard.

But there is another limit while setting the value of clock frequency. That is how fast a logic unit can get out the answer, for instance, if we need 0.5ms to complete an addition operation, then the frequency should be $10.5 \times 10^3 = 2\text{kHz}$. Actually the speed of electrons is always light speed, so the time of operation is reflecting how long the electrons need to travel to get the final value, which is judging the design from the producer.

3.2 Fetch, Decode, Execute Cycle

FETCH

Register Transfer Notation

This is a notion or you may think it as a cycle or process. From coursebook's definition (but I DO NOT suggest you to remember this), is a **short hand notation to show movement of data and instructions in a processor, can be used to represent the operation of the fetch-execute cycle**. You can see the graph below, but I think it is abstract. So I rewrite it in easier words.

```
MAR ← [PC]      contents of PC copied into MAR
PC  ← [PC] + 1  PC is incremented by 1
MDR ← [[MAR]]   data stored at address shown in MAR is
                copied into MDR
CIR ← [MDR]     contents of MDR copied into CIR
```

Notice In the graph above, [] means decode or search what data is in this address.

Register Transfer Notation

You should understand what is transferred, and why. This is used in **fetch** stage, which is to transfer *instructions and data out of* register into the **MDR**. And we know that PC stands for the next instruction address, which after each cycle it needs to add one to itself.

Now we shall begin (IELTs angries!:)D). But a new cycle.

1. PC is set to 26. Assume this is the instruction we need to fetch.
2. From the table we see in address 26, there is LDX. We send 26 through to **MAR** -inside CPU. Then through **Address Bus** 26 is sent to RAM.
3. RAM locate address 26, decode and send data LDX out through **Data Bus** to **MDR**. At the same time, to make CPU not free, PC add itself 1.
4. **MDR** stores LDX, then send it to **CIR**.

Address	Instruction
26	LDX
27	STO
28	ADD
29	LDD
30	ADD
31	CMP
32	JMP
33	OUT
34	LDD
...	...

Decode and Execute

Actually this cycle is not allowed to study at this level. But if you study CS in university, this part is a very difficult part in your major projects. You are welcomed to study this section in the following websites:

- https://zhuanlan.zhihu.com/p/589178906?utm_id=0
- <https://www.youtube.com/playlist?list=PL8dPuuaLjXtNlUrzyH5r6jN9uII>
- https://www.bilibili.com/video/BV1EW41lu7th/?share_source=copy_web

3.3 Interrupts

Definition 3.3.1 (Interrupts). This is **NOT** required to remember but you had better to understand it. Signal sent from a device or software to a processor requesting its attention; the processor suspends all operations until the interrupt has been serviced.

Sometimes A LOT of interrupts are required, but CPU can deal only one at each time, then we should set a priority to check which one is going to service.

Definition 3.3.2 (Interrupt priority). This is also **NOT** required to remember but you had better to understand it. All interrupts are given a priority so that the processor knows which need to be serviced first and which interrupts are to be dealt with quickly.

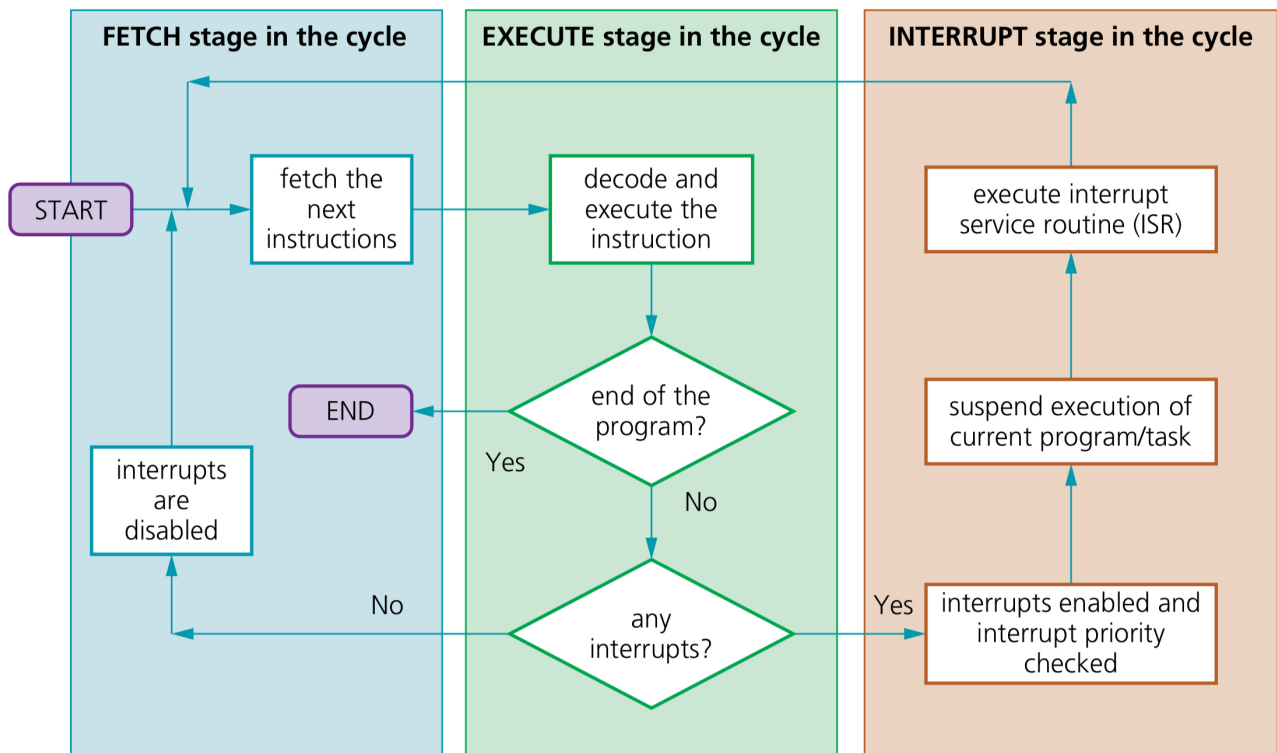


Figure 3.3: Interrupts

Types of interrupts

- » A timing signal.
- » I/O process. *e.g.* A request from printer.
- » A hardware default. An overflow, empty flag, divided by zero or something.
- » Interaction. But personally I think this should belong to I/O process.
- » Software error. Will be extended if you learn software.

3.4 Assembly language

The instructions consist of two parts, **Opcode** and **Operand**. A piece of instruction usually uses a three-capital-alphabet term and a number.

Operand types

ACC accumulator
 num an address
 # a denary number
 & a hex number
 B a bin number
 spc nothing.

△ You should be able to recognise these when they appear.

Code	Rand	Code	Rand	Code	Rand
LDM	#n	MOV	num	CMP	#n
LDD	num	STO	num	SUB	num
LDI	num	END		CMI	num
LDX	num	IN		INC	num
LDR	#n	OUT		DEC	num
LDR	ACC	ADD	num	JMP	num
JPE	num	JPN	num	CMP	num
LSL	#n	LSR	#n	AND	#n
AND	num	XOR	#n	XOR	num
OR	#n	OR	num	NOT	#n

Table 3.1: Instructions

These are the instructions you need to know, but you don't need to remember them, just make sure your eyes are familiar to them, and understand them before exams, or you have to spend time during examinations.

Remember, **not all** the programs start their first instructions at **address 0**. Sometimes it starts at 100 or 25, please pay attention to them!

Those instructions can be divided into several kinds: **MOVEMENT, I/O, CALCULATE, JUMP, COMPARE, SHIFT, LOGIC**.

The most difficult operations include **LDI, LDX, JPE, JPN, CMI**.

Now I will explain them immediately.

Load Modes -LDx

iMmediate LDM. It is followed by a number, put it into ACC immediately.

Direct LDD. It is followed by a number, search for this address and fetch data into ACC directly.

Indirect LDI It is followed by a number, search for this address, fetch data, then if data is a number, search for this number again, then fetch that second data, fetch into ACC indirectly.

index LDX It is followed by a number, add it and the number in IX. Calculate and search for the answer address, fetch into ACC.

example1 LDX 45. IX=23. You should look for address $45 + 23 = 68$.

example2 LDI 67. address 67=34. You should look for 34.

Compare and Jump instructions

Remember, compare means that judging **two values are the same or not**. *Not comparing if A is bigger than B*. If two values are the same, then output **TRUE**, otherwise is **FALSE**. But this value is not stored in any place we have learnt, so you should calculate the result and write it down on your exam paper.

When COMPARE instruction appears, it is followed by a JUMP instruction. JPE is jump to somewhere when compare is true. JPN is jump to somewhere when compare is false. JMP is jump to somewhere no matter what has happened.

```

1 def fv(a,b):
2     list,m,p=[],a,0
3     while m>(b-1):
4         list.append(m%b)
5         m=m//b
6     list.append(m)
7     p=list[len(list)-1]
8     for i in range(1,len(list)):
9         p=p*10+list[-(i+1)]
10    return p
11
12 u=0
13 for n in range(100,1000):
14     s,i=str(fv(n,5)+fv(n,6)),str(n)
15     if i[-2:]==s[-2:]:
16         u+=1
17         print(n,fv(n,5),fv(n,6))
18
19 print(u)

```

Listing 3.1: Python example

Why we do use COMPARE and JUMP?

As you can see this python example, well, some of you may not have learnt this part, you can see that there is a **definition of function**, with a **for loop** and an **if judgement**. While using the function, PC jumps to the address of the start of the function, that is **JMP** in assembly language. While steps into a loop, especially when a loop comes to an end, PC jumps to the address of the start of the loop, also **JMP** in assembly language. However, in if judgment, in this case, in line 15-17, it compares whether **i[-2:]==s[-2:]**, if so, then jumps to **u+=1**, if not, jumps to the end of the loop, also starts the next generation of the loop, which uses **CMP**, **JPE** and **JPN**.

Declaration This program is written by myself in Sep. 2023, nothing use for CS learning but I think it can explain something. Copyright @JTST.

Chapter 4

System Software

4.1 Operating System

Definition 4.1.1 (OS). Operating System. A software platform that provides facilities for programs to be run which are of benefit for the user.

Tasks of OS

- Management
 1. Process
 2. Memory
 3. Storage
 4. Hardware/Device
 5. Security
 6. File
- Interface
 1. User
 2. Application

Memory Management

It has three main tasks: memory *optimisation*, *organisation* and *protection*.

- Protection is to protect the memory from occupied by another process, and cannot directly read the data by any process.
- Organisation is best use of the space of memory, maximise the percentage of storage used, sometimes may use virtual memory.
- Optimisation is tracking all the address of memory used by all processes.

4.2 Utility Software, Libraries, and Translators

Definition 4.2.1 (Utility program). Parts of the operating system that carrying out certain functions.

Examples

- » **Hard disk formatter.** If your disk is full and want to use in the future, it would be a nice choice for you to format it, which is clear all the data and initialise.
- » **Virus checker.** Identifies viruses and gets rid of them.
- » **Defragmenter software.** Part of a disk repair utility program, but used for managing the space in the disk. Not continuously.
- » **File compression.** See chapter 1 -compression. Meaning on paper.
- » **Backup software.** Make three copies of a file while they are in security mode.

Definition 4.2.2 (Program Library). A library on computer where programs and routines are stored which can be freely accessed by other software developers for use in their own programs.

Pros of using DLL files	Cons of using DLL files
the executable code of the main program is much smaller since DLL files are only loaded into memory at run time	the executable code is not self-contained, therefore all DLL files need to be available at run time otherwise error messages (such as missing .dll error) will be generated and the software may even crash
it is possible to make changes to DLL files independently of the main program, consequently if any changes are made to the DLL files it will not be necessary to recompile the main program	any DLL linking software in the main program needs to be available at run time to allow links with DLL files to be made
DLL files can be made available to a number of applications at the same time	if any of the DLL files have been changed (either intentionally or through corruption) this could lead to the main program giving unexpected results or even crashing
all of the above save memory and also save execution time	malicious changes to DLL files could be due to the result of malware, thus presenting a risk to the main program following the linking process

Figure 4.1: Pros and Cons of .dll

Definition 4.2.3 (Assembler). It translates **assembly language** into **machine code**. It performs a 1:1 translation from mnemonic to a direct instruction.

Definition 4.2.4 (Compiler). It translates the whole source **high level** program into **object code**(.obj) that can be stored and re-used.

Definition 4.2.5 (Interpreter). It translates and executes a program **line by line**. No object code, translate each time.

Compiler	Interpreter
The end user only needs the executable code, therefore, the end user benefits as there is no need to purchase a compiler to translate the program before it is used.	The end user will need to purchase a compiler or an interpreter to translate the source code before it is used.
The developer keeps hold of the source code, so it cannot be altered or extended by the end user, therefore, the developer benefits as they can charge for upgrades and alterations.	The developer relinquishes control of the source code, making it more difficult to charge for upgrades and alterations. Since end users can view the source code, they could potentially use the developer's intellectual property.
Compiled programs take a shorter time to execute as translation has already been completed and the machine code generated may have been optimised by the compiler.	An interpreted program can take longer to execute than the same program when compiled, since each line of the source code needs to be translated before it is executed every time the program is run.
Compiled programs have no syntax or semantic errors.	Interpreted programs may still contain syntax or semantic errors if any part of the program has not been fully tested, these errors will need to be debugged.
The source program can be translated on one type of computer then executed on another type of computer.	Interpreted programs cannot be interpreted on one type of computer and run on another type of computer.
A compiler finds all errors in a program. One error detected can mean that the compiler finds other dependent errors later on in the program that will not be errors when the first error is corrected. Therefore, the number of errors found may be more than the actual number of errors.	It is easier to develop and debug a program using an interpreter as errors can be corrected on each line and the program restarted from that place, enabling the programmer to easily learn from any errors.
Untested programs with errors may cause the computer to crash.	Untested programs should not be able to cause the computer to crash.
The developer needs to write special routines in order to view partial results during development, making it more difficult to assess the quality of particular sections of code.	Partial results can be viewed during development, enabling the developer to make informed decisions about a section of code, for example whether to continue, modify, or scrap and start again.
End users do not have access to the source code and the run-time libraries, meaning they are unable to make modifications and are reliant on the developer for updates and alterations.	If an interpreted program is purchased, end users have all the source code and the run-time libraries, enabling the program to be modified as required without further purchase.

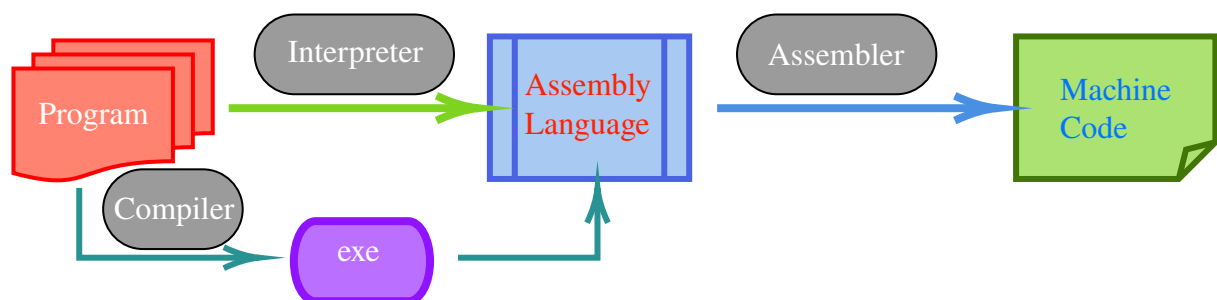
Figure 4.2: **Pros** and **Cons** between Compiler and Interpreter

Figure 4.3: Flowchart of translating languages

ENDS HERE BEFORE MID-TERM EXAMINATIONS